

/plugins

Plugin management: `/plugins list [--verbose] | add [--force] | remove | enable | disable | test | codex [query] [--category ] [--refresh] | claude [query] [--category ] [--refresh] | stores [--check] | help`

Overview

`/plugins` manages third-party plugins written for OpenAI Codex and Claude Code/Desktop. Plugin JavaScript runs on an embedded, budget-sandboxed engine (`rquickjs`) inside the `ragent` binary - no external Node.js, Deno, or Bun installation is required. Enabled plugins contribute tools (registered as `plugin_<id>_<tool>`) and slash commands through a versioned `ragent` host API.

A bare `/plugins`, `/plugins help`, and any unrecognised subcommand all print the same usage block and create or modify no files.

The same operations are available from a shell via the `ragent plugins <sub>` CLI parity surface; the two surfaces share one help text and one set of command handlers.

Syntax

```
/plugins list [--verbose]      # list discovered plugins
/plugins list --mcp            # live MCP tool inventory (servers, status, tools)
/plugins add <source> [--force] # install a plugin (enabled on install)
/plugins remove <pluginid>     # uninstall (refused while enabled)
/plugins enable <pluginid>     # enable, load, register tools/commands
/plugins disable <pluginid>    # unload, deregister tools/commands
/plugins test <pluginid>       # isolated harness: load + invoke each tool once
/plugins codex [query] [--category <name>] [--refresh] # browse the Codex marketplace
/plugins claude [query] [--category <name>] [--refresh] # browse the Claude marketplace
/plugins stores [--check]      # report each store's endpoint and provenance
/plugins help                  # usage block
```

Options / Subcommands

Form	Description
<code>/plugins list [--verbose]</code>	One row per discovered plugin showing id, name, version, dialect, state (disabled/enabled/loaded/errored), and contributed tool/command names, plus MCP / MCP Tools server and tool counts, and a totals summary. <code>--verbose</code> (or <code>-v</code>) appends per-plugin telemetry counters.
<code>/plugins list --mcp</code>	Print the live MCP tool inventory: every connected MCP server (including those bridged from a plugin's <code>mcpServers</code> section) with its status, tool count, and each tool's registry name (<code><tool> -> mcp_<server>_<tool></code>).

Form	Description
<code>/plugins add <source> [--force]</code>	Install a plugin and validate its manifest; reports the plugin id and dialect. The plugin is recorded enabled and loads at the next session start; turn it off with <code>/plugins disable</code> . Refuses a duplicate id unless <code>--force</code> is given.
<code>/plugins remove <pluginid></code>	Uninstall a plugin from the store. Refused while the plugin is enabled.
<code>/plugins enable <pluginid></code>	Mark the plugin enabled, load it into the current session, and register its tools and commands. Reports the declared permissions and the load outcome.
<code>/plugins disable <pluginid></code>	Mark the plugin disabled, unload it, and deregister every tool and command it contributed, confirming how many of each were removed. Files are not deleted.
<code>/plugins test <pluginid></code>	Load the plugin in an isolated harness (never touching the live session), invoke each contributed tool once with schema-derived sample arguments, and report per-step [ok]/[fail] results with wall-clock time.
<code>/plugins codex [query] [--category <name>] [--refresh]</code>	Open the interactive Codex marketplace browser over the store index: <code>query</code> pre-fills the search field, <code>--category</code> pre-selects a left-hand category row, and <code>--refresh</code> bypasses on-disk store cache. The left-hand navigator lists ALL plus the distinct categories and tags the index declares (Tab focus, Up/Down move, Enter apply, c clear, or click/scroll with the mouse).
<code>/plugins claude [query] [--category <name>] [--refresh]</code>	Open the interactive Claude marketplace browser, with the same query, <code>--category</code> , and <code>--refresh</code> arguments.
<code>/plugins stores [--check]</code>	Report each store's effective endpoint and its source; <code>--check</code> also contacts each store and reports availability and plugin count.
<code>/plugins help</code>	Print the usage block. A bare <code>/plugins</code> or an unknown subcommand does the same.

Sources accepted by `/plugins add`

- a local directory containing a plugin manifest;
- a local `.zip` or `.tar.gz` package file;
- an `https://` URL pointing at a `.zip/.tar.gz` package (non-https URLs are refused).

Examples

List everything with telemetry:

```
/plugins list --verbose
```

Install a plugin from a local directory and enable it:

```
/plugins add ./my-plugin  
/plugins enable my-plugin
```

Test a plugin without touching the live session:

```
/plugins test my-plugin
```

Unload without deleting files:

```
/plugins disable my-plugin
```

Output

Reports are prefixed with a From: `/plugins <sub>` attribution line. `list` renders a fixed-width table with one count column per contribution kind (Tools, Commands, Skills, Agents, Hooks) plus two MCP columns - MCP (the number of MCP servers the plugin declares) and MCP Tools (the total tools those servers advertise, summed from the live MCP client; ? when a server has not connected, never 0):

```
| ID |Name |Version |Dialect |State      |Tools|Commands|Skills|Agents|Hooks|MCP|MCP Tools|
```

followed by Contributions:, Unsupported capabilities:, and Errors: sections — the Contributions: block lists the detailed names for each kind, e.g. skills [db-setup]; agents [agents/security-reviewer.md]; hooks [PreToolUse, SessionStart]; mcp [mongodb.mongodb (35 tools)] (1 server(s), 35 tool(s)) — and a summary line (Total: N plugin(s) - X enabled, X disabled, X errored, with loaded plugins counted as enabled).

`test` renders one line per harness step:

```
[ ok ] discovery (2 ms)  
[ ok ] manifest (0 ms)  
[ ok ] entry (5 ms)  
[ ok ] tool ping (1 ms)
```

and ends by confirming the harness was unloaded with the live session untouched.

Configuration

The `plugins` block in `ragent.json` controls the subsystem (`enabled`, `max_execution_ms`, `max_entry_ms`, `max_memory_mb`, `store_dir`, and per-plugin permissions). With `plugins.enabled: false`, every subcommand other than `help` reports the subsystem is disabled and no plugin code executes. Plugins are discovered under `.ragent/plugins/` (project), falling back to `~/.config/ragent/plugins/` (user-global).

See `docs/howtos/config.md §7.35` and `specs/plugins/SPEC.md`.

Related

- `docs/howtos/plugins.md` - the full plugin-system manual
- `/reload` - reload customizations after plugin files change
- `/tools` - toggle tool visibility (plugin tools are always advertised while enabled)
- `ragent plugins <sub>` - CLI parity for the same subcommands
- `specs/plugins/SPEC.md` - the full plugin-system specification